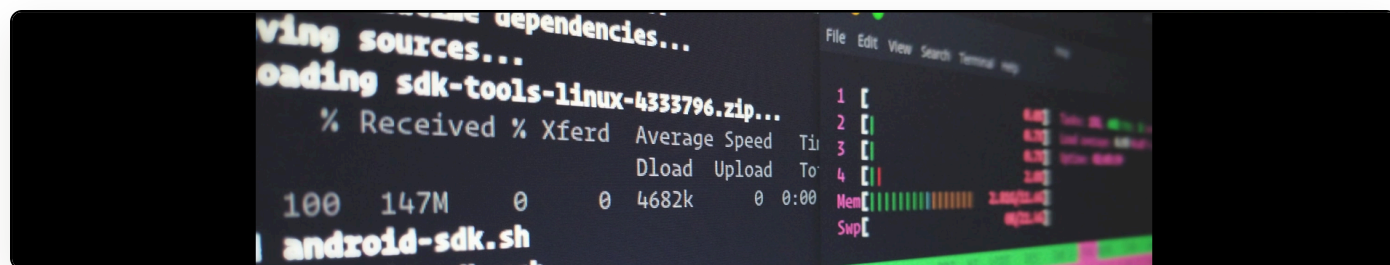


Tied to the mast? A response to RMS's principles for loyal computers

Cite as: Martin Paul Eve, "Tied to the mast? A response to RMS's principles for loyal computers", *eve.gd: Martin Paul Eve*, December 19, 2014, <https://doi.org/10.59348/erzpp-y8227>.



In a recent essay, Richard M. Stallman, pioneer of the free software movement, asked “[what does it mean for a computer to be loyal?](#)” The "tentative definition" that Stallman outlines consists of: Neutrality towards software; Neutrality towards protocols; Neutrality towards implementations; Neutrality towards data communicated; Debugability; Documentation; and Completeness. I won't, here, reproduce the whole essay as you can read it on the GNU site. What I want to do instead is to point out a few ambiguities and potential sticking points that I identified in the definition here, mostly surrounding the principle of remote attestation.

Hardware vs. Software

The first thing that struck me about the "disloyal computer" definition is that although it is generally sound and speaking of hardware, it also seems, at various points, to be speaking about software or the combination of software and hardware. This may not be the intention of the definition. The principles of "neutrality towards software", for example, seem designed to ensure that there is no in-built hardware/firmware signature checking (as in the case of Android smartphones) that will prevent the user from installing whatever operating system or software they would like. The same could be said of some UEFI boot systems ("secure boot").

However, when the piece goes on to talk about remote attestation (i.e. being able to remotely identify which version of software/implementation is being run at the client side), things are, at least to me, a little murkier. Stallman writes:

This entails that the computer rejects remote attestation, that is, that it does not permit other computers to determine over the network whether your computer is running one particular software load. Remote attestation gives web sites the power to compel you to connect to them only through an application with DRM that you can't break, denying you effective control over the software you use to communicate with them. Netflix is a notorious example of this.

If this is supposed to refer to the hardware level, then great, that's a solid principle. However, the majority of remote attestation implementations are contained within non-free software or in uninspectable remote implementations (hardware + software), Netflix, indeed, being the best example. Is the "computer" in this scenario referring only the hardware or to the whole environment that makes up the computation platform? If hardware runs code as part of its own, integral operation ("firmware"), how does that sit within this definition?

Design-time vs. Run-time

The second point, following on from this, that occurs to me is the different points of design in the system. Let us suppose that we designed a loyal hardware system but that the user wishes to run Netflix. In this environment, two of the principles might come into contradiction with each other:

The computer will run, without prejudice, whatever software you install in it, and let that software do whatever its code says to do.

If a computer allows web sites to bar you from using a modified program with them, it is loyal to them, not to you.

Principle one: the user wants to run code that will enable remote attestation. Principle two: the "computer", as a combination of hardware and software, would then be allowing remote attestation.

Like Odysseus and the sirens, if the design states that the users should be tied to the mast, is it disloyal to its users when they change their mind at a later, run-time, state, or should the earlier, design-time instruction for loyalty, stand? Should the computer bar code from running that would make the system, as a whole, behave in a disloyal way, or is it the right of users to tell their own systems to turn against them?

Precedence of principles

In other words: which of these principles is strongest? Asimov recognised this in his "laws" of robotics. These laws are:

1. A robot may not injure a human being or, through inaction, allow a human being to come to harm.
2. A robot must obey the orders given to it by human beings, except where such orders would conflict with the First Law.
3. A robot must protect its own existence as long as such protection does not conflict with the First or Second Law.

In this setup, it is recognised that it is possible for the principles to come into contradiction with each other and an order of precedence is set. I think that the definitions for loyalty set here would also benefit from precedence. Which is stronger? "The computer will run, without prejudice, whatever software you install in it, and let that software do whatever its code says to do." or the anti-Tivoization principle of blocking remote attestation.

Conclusions

Much of this definition is sound but I think the remote attestation parts are weakest and need clarifying. In particular:

1. More clearly define where this remote attestation is happening (i.e. at hardware, firmware or software levels) and what is allowed.
2. Outline whether the point of this definition is to restrict modes of operation that would allow the user to command the system to become disloyal.
3. Set an order of precedence for the principles of loyalty to avoid contradictory situations, if not resolved by #1.